
Integer and Matrix Multiplication

Long Multiplication

High School method for multiplying integers

Example: 163×97

			1	6	3
		x		9	7
		1	1	4	1
	1	4	6	7	
	1	5	8	1	1

Binary Long Multiplication

Multiply. Given two n -bit integers a and b , compute $a \cdot b$.

- Example: 163×97 , i.e., 10100011×01100001

```
      1 0 1 0 0 0 1 1
    × 0 1 1 0 0 0 0 1
    -----
          1 0 1 0 0 0 1 1
        0 0 0 0 0 0 0 0
        0 0 0 0 0 0 0 0
        0 0 0 0 0 0 0 0
        0 0 0 0 0 0 0 0
        0 0 0 0 0 0 0 0
      1 0 1 0 0 0 1 1
      1 0 1 0 0 0 1 1
    0 0 0 0 0 0 0 0
    -----
    0 1 1 1 1 0 1 1 1 0 0 0 0 1 1
```

Cost. n binary multiplications to generate each line; we generate n lines. Thus, total cost $\Theta(n^2)$ multiplications (plus $\Theta(n^2)$ binary additions because we summarize n lines)

Binary Multiplication: Break into smaller problems

Goal. Given two n -bit integers a and b , compute: $a \cdot b$.

- Example: 163×97 , i.e., 10100011×01100001 ($n=8$)

Rewrite numbers. $a = 2^{n/2}a_1 + a_0$, $b = 2^{n/2}b_1 + b_0$

$a_1=1010$, $a_0=0011$

$b_1=0110$, $b_0=0001$

Note: a_1 and b_1 can be thought of having $n/2$ (4 in this example) least significant bits that are equal to 0.

Given rewritten numbers: $a \cdot b = (2^{n/2}a_1 + a_0) \cdot (2^{n/2}b_1 + b_0) = 2^n a_1 b_1 + 2^{n/2}(a_1 b_0 + a_0 b_1) + a_0 b_0$

Observation: Multiplication by 2^k can be done in one time unit by a left shift of k bits.

- **Example:** $12=00001100$. $12 \times 2^3=96$ is the same as left shifting 00001100 by 3 bits
- $00001100 \ll 3 = 01100000 = 96$
- We use $\ll$ to denote left shifting

Binary Multiplication: Motivation of D&C

Instead of multiplying two n -bit integers a and b directly with long multiplication:

1] Rewrite numbers: $a = 2^{n/2}a_1 + a_0$, $b = 2^{n/2}b_1 + b_0$

$$a_1 = 1010, a_0 = 0011$$

$$b_1 = 0110, b_0 = 0001$$

2] The product to be computed becomes:

$$\begin{aligned} a \cdot b \\ = (2^{n/2}a_1 + a_0) \cdot (2^{n/2}b_1 + b_0) = 2^n a_1 b_1 + 2^{n/2}(a_1 b_0 + a_0 b_1) + a_0 b_0 \end{aligned}$$

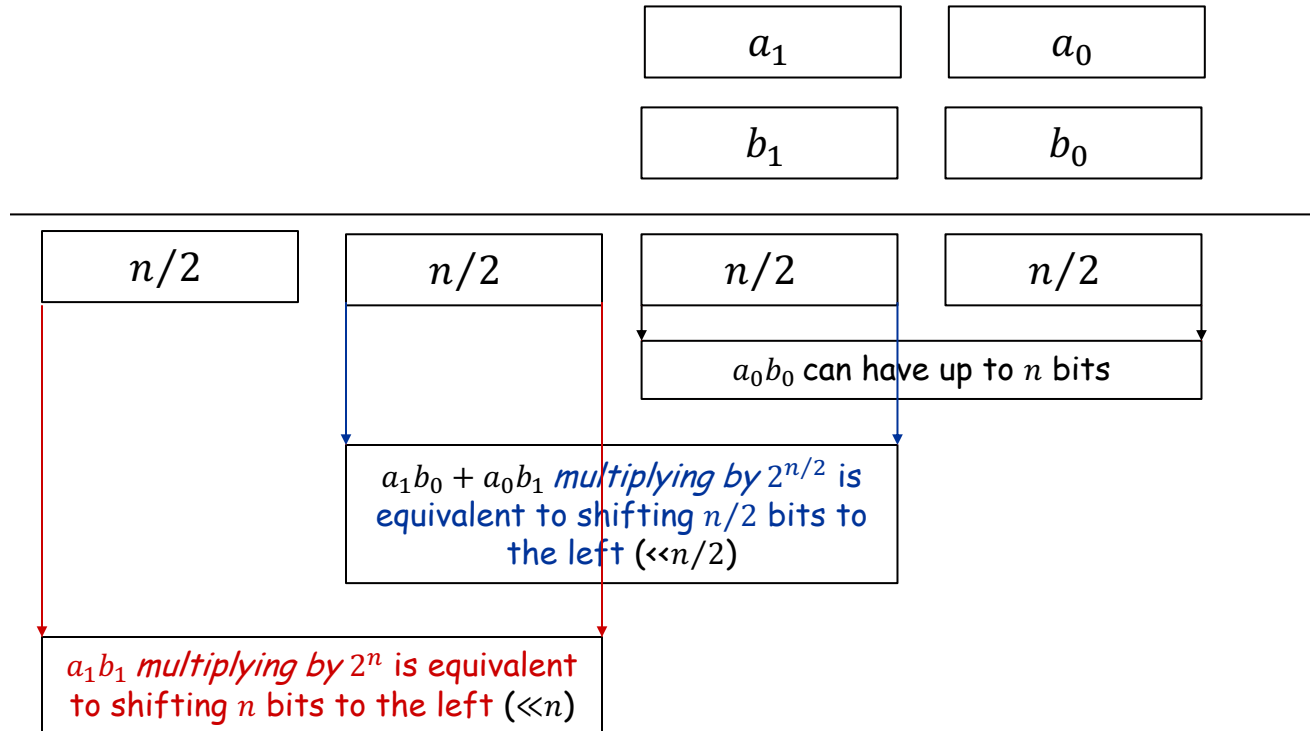
3] The new computation requires 4 products of integers, each with $n/2$ bits:

$$a_1 b_1, a_1 b_0, a_0 b_1, a_0 b_0$$

4] Apply D&C by splitting a problem of size n , to 4 problems of size $n/2$.

D&C Binary Multiplication: Visualization

$$a \cdot b = 2^n a_1 b_1 + 2^{n/2} (a_1 b_0 + a_0 b_1) + a_0 b_0$$



The first divide-and-conquer algorithm for integer multiplication

Suppose the bits are stored in arrays $A[1..n]$ and $B[1..n]$, $A[1]$ and $B[1]$ are the least significant bits

Multiply(A, B) :

$n \leftarrow \text{size of } A$

if $n = 1$ **then return** $A[1] \cdot B[1]$

$mid \leftarrow \lfloor n/2 \rfloor$

$U \leftarrow \text{Multiply}(A[mid + 1..n], B[mid + 1..n])$ // $a_1 b_1$

$V \leftarrow \text{Multiply}(A[mid + 1..n], B[1..mid])$ // $a_1 b_0$

$W \leftarrow \text{Multiply}(A[1..mid], B[mid + 1..n])$ // $a_0 b_1$

$Z \leftarrow \text{Multiply}(A[1..mid], B[1..mid])$ // $a_0 b_0$

$M[1..2n] \leftarrow 0$

$M[1..n] \leftarrow Z$ // $a_0 b_0$

$M[mid + 1..] \leftarrow M[mid + 1..] \oplus V \oplus W$ // $+ (a_1 b_0 + a_0 b_1) \ll n/2$

$M[2mid + 1..] \leftarrow M[2mid + 1..] \oplus U$ // $+ a_1 b_1 \ll n$

return M

$\oplus$: denotes the integer addition algorithm

Analysis with Expansion Method

Recurrence.

For, $n > 1$, $T(n) = 4T(n/2) + n$. $T(1) = 1$

$$T(n) = 4 T\left(\frac{n}{2}\right) + n$$

$$= 4 \left(4 T\left(\frac{n}{4}\right) + \frac{n}{2} \right) + n$$

$$= 4^2 T\left(\frac{n}{2^2}\right) + 2n + n$$

$$= 4^2 \left(4 T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \right) + 2n + n$$

$$= 4^3 T\left(\frac{n}{2^3}\right) + 2^2n + 2n + n$$

$$= 4^3 \left(4 T\left(\frac{n}{2^4}\right) + \frac{n}{2^3} \right) + 2^2n + 2n + n$$

$$= 4^4 T\left(\frac{n}{2^4}\right) + (2^3 + 2^2 + 2 + 1)n$$

....

$$= 4^i T\left(\frac{n}{2^i}\right) + (2^{i-1} + \dots + 2 + 1)n$$

Analysis with Expansion Method (cont)

When we reach level i we have (total) cost:

$$4^i T\left(\frac{n}{2^i}\right) + (2^{i-1} + \dots + 2 + 1)n$$

We stop when the problem size becomes 1, i.e., when we reach level i , such that: $n/2^i = 1 \Rightarrow n = 2^i \Rightarrow i = \log_2 n$. Thus, $4^i = 4^{\log_2 n} = n^{\log_2 4} = n^2$ and $T(1)=1$. The total cost becomes

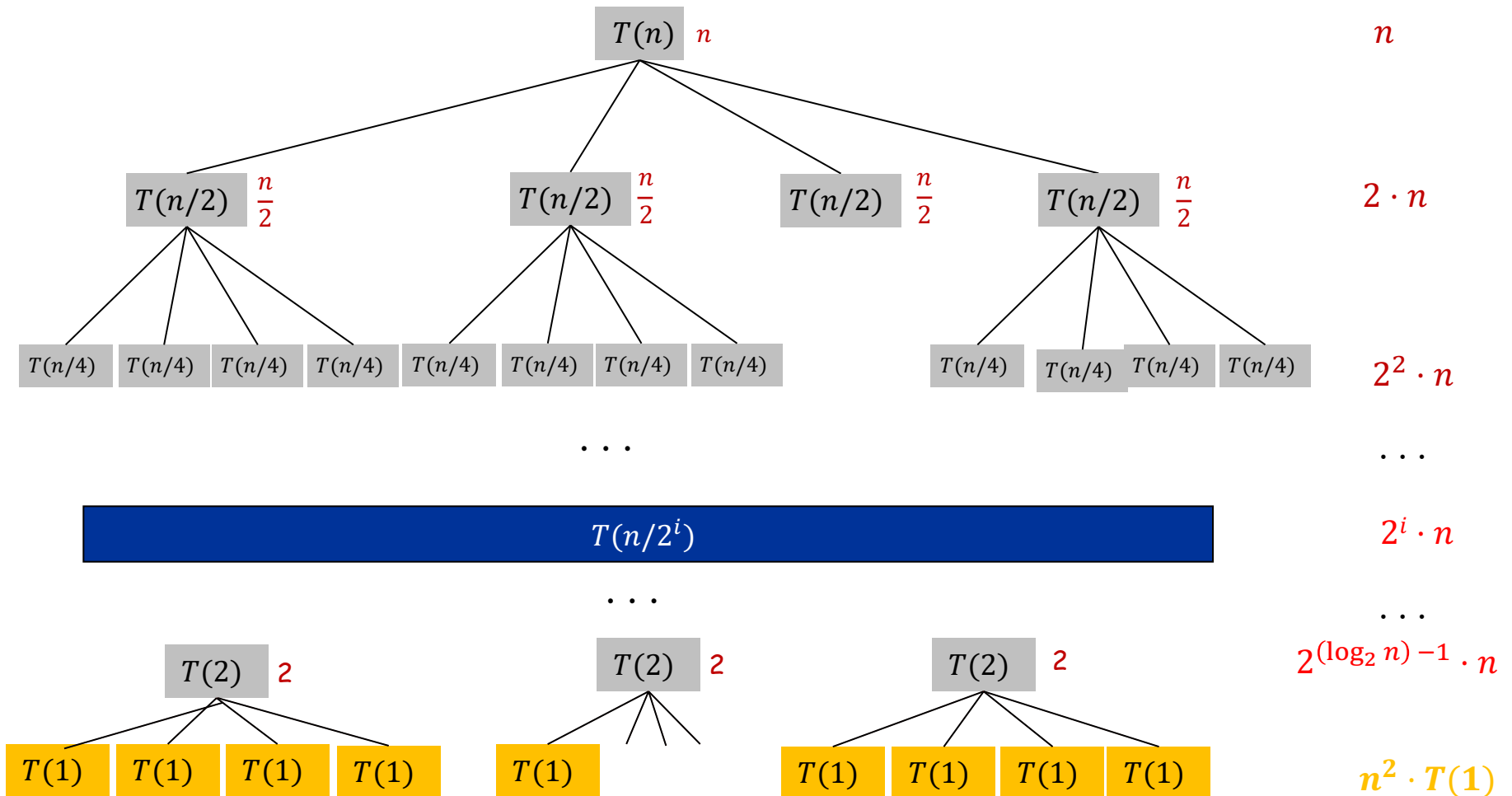
$$\begin{aligned} & n^2 \\ & + n(2^{i-1} + \dots + 2 + 1) = n^2 + n \sum_{i=0}^{\log_2 n - 1} 2^i = n^2 + n \frac{2^{\log_2 n} - 1}{2 - 1} = \\ & = n^2 + n(n - 1) = \Theta(n^2) \end{aligned}$$

Analysis with Recursion Tree Method

Recurrence:

$$T(n) = 4T(n/2) + n; \quad T(1) = 1$$

Solve the recurrence:



Analysis with Recursion Tree Method (cont)

$$\begin{aligned} & n + 2n + 2^2n + \dots + 2^{(\log_2 n)-1}n + 4^{\log_2 n} T(1) \\ &= \\ & n(1 + 2 + 2^2 + 2^3 + \dots + 2^{(\log_2 n)-1}) + n^2 = \\ & n \left(\frac{2^{\log_2 n} - 1}{2 - 1} \right) + n^2 = n(n - 1) + n^2 = \Theta(n^2) \end{aligned}$$

- The divide-and-conquer algorithm is as bad as the primary school method
- Essentially, the algorithm still multiplies every bit of A with every bit of B .
- Compared with merge sort, the key difference is that one problem generates **4** subproblems of size **$n/2$** .

Karatsuba Multiplication

- Let $a = a_1 2^{n/2} + a_0$, and $b = b_1 2^{n/2} + b_0$
where a_1, a_0, b_1, b_0 are all $(n/2)$ -bit integers.

- We already saw

$$ab = a_1 b_1 2^n + (a_1 b_0 + a_0 b_1) 2^{n/2} + a_0 b_0$$

- Observation: We do not need the individual products $a_1 b_0, a_0 b_1$. Only their sum $a_1 b_0 + a_0 b_1$.
- But given that we compute $a_1 b_1, a_0 b_0$ anyway, this sum requires only one additional multiplication:

$$a_1 b_0 + a_0 b_1 = (a_1 + a_0)(b_1 + b_0) - a_1 b_1 - a_0 b_0$$

Calculating ab now only requires performing **3** multiplication subproblems of size $n/2$!

Karatsuba's multiplication algorithm

Multiply(A, B) :

$n \leftarrow \text{size of } A$

if $n = 1$ **then return** $A[1] \cdot B[1]$

$mid \leftarrow \lfloor n/2 \rfloor$

$U \leftarrow \text{Multiply}(A[mid + 1..n], B[mid + 1..n])$

// $a_1 b_1$

$Z \leftarrow \text{Multiply}(A[1..mid], B[1..mid])$

// $a_0 b_0$

$A' \leftarrow A[mid + 1..n] \oplus A[1..mid]$

// $a_1 + a_0$

$B' \leftarrow B[mid + 1..n] \oplus B[1..mid]$

// $b_1 + b_0$

$Y \leftarrow \text{Multiply}(A', B')$

// $(a_1 + a_0)(b_1 + b_0)$

$M[1..2n] \leftarrow 0$

$M[1..n] \leftarrow M[1..n] \oplus Z$

// $a_0 b_0$

$M[mid + 1..] \leftarrow M[mid + 1..] \oplus Y \ominus U \ominus Z$

// $+(a_1 b_0 + a_0 b_1) \ll n/2$

$M[2mid + 1..] \leftarrow M[2mid + 1..] \oplus U$

// $+ a_1 b_1 \ll n$

return M

$\oplus \ominus$: denotes the integer addition/subtraction algorithm

Analysis with Expansion Method

Recurrence.

For, $n > 1$, $T(n) = 3T(n/2) + n$. $T(1) = 1$

$$T(n) = 3 T\left(\frac{n}{2}\right) + n$$

$$= 3 \left(3T\left(\frac{n}{4}\right) + \frac{n}{2} \right) + n$$

$$= 3^2 T\left(\frac{n}{2^2}\right) + \frac{3}{2}n + n$$

$$= 3^2 \left(3T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \right) + \frac{3}{2}n + n$$

$$= 3^3 T\left(\frac{n}{2^3}\right) + \frac{3^2}{2^2}n + \frac{3}{2}n + n$$

$$= 3^3 \left(3T\left(\frac{n}{2^4}\right) + \frac{n}{2^3} \right) + \frac{3^2}{2^2}n + \frac{3}{2}n + n$$

$$= 3^4 T\left(\frac{n}{2^4}\right) + \left(\frac{3^3}{2^3} + \frac{3^2}{2^2} + \frac{3}{2} + 1 \right) n$$

....

$$= 3^i T\left(\frac{n}{2^i}\right) + \left(\frac{3^{i-1}}{2^{i-1}} + \frac{3^{i-2}}{2^{i-2}} + \cdots + \frac{3}{2} + 1 \right) n$$

Analysis with Expansion Method (cont)

When we reach level i we have (total) cost:

$$3^i T\left(\frac{n}{2^i}\right) + \left(\left(\frac{3}{2}\right)^{i-1} + \dots + \left(\frac{3}{2}\right) + 1\right)n$$

We stop when the problem size becomes 1, i.e., when we reach level i , such that: $n/2^i = 1 \Rightarrow n = 2^i \Rightarrow i = \log_2 n$. Thus, $3^i = 3^{\log_2 n} = n^{\log_2 3} = n^{1.585}$ and $T(1)=1$. The total cost is:

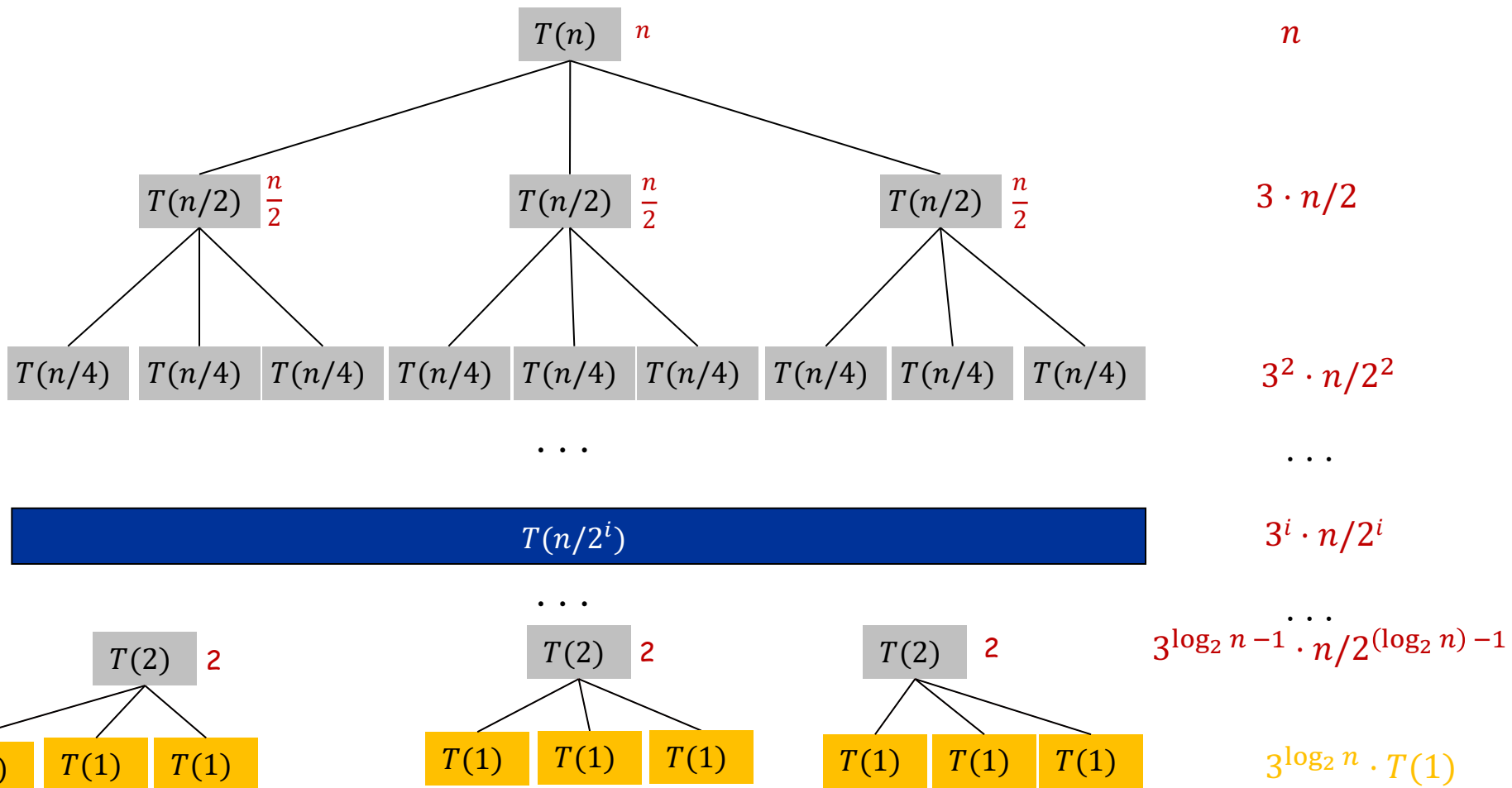
$$n^{\log_2 3} + n \left(\left(\frac{3}{2}\right)^{i-1} + \dots + \frac{3}{2} + 1 \right) =$$
$$\Theta(n^{\log_2 3}) = \Theta(n^{1.585\dots})$$

Analysis with Recursion Tree Method

Recurrence:

$$T(n) = 3T(n/2) + n$$

Solve the recurrence:



Analysis (continued)

Recurrence For First D&C Algorithm

$$T(n) = 4T(n/2) + n; \quad T(1) = 1$$

Solution: $T(n) = \Theta(n^2)$

Recurrence For Karatsuba Multiplication

$$T(n) = 3T(n/2) + n; \quad T(1) = 1$$

Solution: $T(n) = \Theta(n^{1.585...})$

Analysis (continued)

Karatsuba Multiplication:

- Dividing each integer into 2 parts, and solve 3 subproblems
 - $T(n) = 3T(n/2) + n, T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585\dots})$

Progressive improvements:

- Dividing each integer into 3 parts, and solve 5 subproblems
 - $T(n) = 5T(n/3) + n, T(n) = \Theta(n^{\log_3 5}) = \Theta(n^{1.465})$
- Dividing each integer into 4 parts, and solve 7 subproblems
 - $T(n) = 7T(n/4) + n, T(n) = \Theta(n^{\log_4 7}) = \Theta(n^{1.404})$
- ...
- An $\Theta(n \log n \log \log n)$ algorithm (based on Fast Fourier Transform)
- An $\Theta(n \log n \log \log \log n)$ algorithm
- An $\Theta(n \log n 2^{\Theta(\log^* n)})$ algorithm ($\log^* n$ is a VERY slow growing function)
- The conjecture was that the problem can be solved in $O(n \log n)$ time.

Conjecture Proven (2019)

- $\Theta(n \log n)$ time algorithm found